**Marevlo Research** / AI Foundations • Module 05 – Batching • Topic 4

# SLO-Aware Scheduling, Worked Out by Hand

The utilization wall and the preemption threshold — two numbers that size a serving cluster

**What this document does.** Batching and speculation set how *fast* the GPU serves. Scheduling decides *which* request goes next and *whether the cluster holds* under load. This document works two pieces of operational math from first principles. First, the **utilization wall**: the M/M/1 expected wait $\mathrm{E}[W] = t_{\mathrm{svc}}\,\rho/(1-\rho)$ blows up as $\rho \to 1$, and we find exactly the utilization at which FIFO breaks the *Nexus* interactive SLO. Second, the **preemption threshold**: when the in-flight KV demand approaches the KV pool, the scheduler is forced to evict. Every figure is reproduced by `m05_t04_scheduling_queue_engine.py` (Appendix A).

**The one idea.** Both failures are *capacity* cliffs, not policy bugs. Queueing delay grows hyperbolically in $\rho$, and KV pressure grows linearly in context length; each has a sharp threshold you can compute before you deploy.

Concepts covered, in order
the M/M/1 wait formula; why $\rho/(1-\rho)$ explodes; the $\rho \to 1$ sweep; solving for the SLO-breaking $\rho$; the utilization wall near $\rho \sim 0.7$–$0.8$; the preemption threshold; the expected-KV-demand formula; and a context-length stress sweep.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

## Contents

## 1 The system under study

The *Nexus* cluster mixes interactive checkout traffic (tight SLO) with offline batch jobs (loose SLO) on the same GPUs. We model the interactive class as an M/M/1 queue and the KV pool as a fixed memory budget.

| Quantity | Symbol | Value here |
|---|---|---|
| Service capacity | $\mu$ | 1000 req/s |
| Mean service time (interactive) | $t_{\text{svc}}$ | 60 ms |
| Interactive TTFT SLO | SLO | 200 ms |
| Max concurrent sequences | $S_{\max}$ | 64 |
| Mean context length | $\mathrm{E}[T_{\text{ctx}}]$ | 2000 tokens |
| KV per token (GQA-8) | $\text{kv}_{\text{tok}}$ | 320 KB |
| KV pool (cluster-wide) | $M_{\text{kv}}$ | 100 GB |

Utilization is $\rho = \lambda/\mu$, the fraction of capacity the arrival rate $\lambda$ consumes. The KV-per-token figure (320 KB) is the same GQA-8 value derived in Topic 1: $2 \times 8 \times 128 \times 80 \times 2\,\text{B}$.

**Intuition**

A queue is a waiting room with one server. If customers arrive almost as fast as the server can clear them ($\rho$ near 1), the room fills faster than it drains and the wait grows without bound. The KV pool is a different room — a coat-check with a fixed number of hooks. When more coats arrive than there are hooks, something gets bumped. Scheduling is managing both rooms at once.

## 2 Step 1 — The M/M/1 expected wait

For a single-server queue with Poisson arrivals (rate $\lambda$) and exponential service (rate $\mu$), the expected time a request spends *waiting* before service is the standard M/M/1 result.

**M/M/1 expected wait**

$$\rho = \frac{\lambda}{\mu}, \qquad \mathrm{E}[W] = t_{\text{svc}} \cdot \frac{\rho}{1-\rho}, \qquad \text{TTFT} = \mathrm{E}[W] + t_{\text{svc}} = \frac{t_{\text{svc}}}{1-\rho}.$$

**Why the wait is hyperbolic in $\rho$**

The factor $\rho/(1-\rho)$ is the queue's amplifier. As $\rho$ rises toward 1 the denominator $1-\rho$ shrinks toward 0, so the wait grows *hyperbolically*, not linearly:

$$\rho = 0.5 \Rightarrow \tfrac{\rho}{1-\rho} = 1, \quad \rho = 0.9 \Rightarrow 9, \quad \rho = 0.99 \Rightarrow 99.$$

The last 10% of utilization ($0.9 \to 0.99$) multiplies the wait by $11\times$. There is no graceful degradation near saturation — the wall is vertical.

## 3 Step 2 — The utilization wall, swept

Plugging $t_{\text{svc}} = 60\,\text{ms}$ into $\mathrm{E}[W]$ and $\text{TTFT} = \mathrm{E}[W] + t_{\text{svc}}$ against the 200 ms SLO:

| $\rho$ | $\lambda$ (req/s) | $\rho/(1-\rho)$ | E[$W$] (ms) | TTFT (ms) | SLO? |
|---|---|---|---|---|---|
| 0.50 | 500 | 1.00 | 60.0 | 120.0 | ✓ OK |
| 0.70 | 700 | 2.33 | 140.0 | 200.0 | ✓ OK (edge) |
| 0.75 | 750 | 3.00 | 180.0 | 240.0 | MISS |
| 0.80 | 800 | 4.00 | 240.0 | 300.0 | MISS |
| 0.90 | 900 | 9.00 | 540.0 | 600.0 | MISS |
| 0.95 | 950 | 19.00 | 1140.0 | 1200.0 | MISS |
| 0.99 | 990 | 99.00 | 5940.0 | 6000.0 | MISS |

At $\rho = 0.50$ the system is comfortable (120 ms TTFT). By $\rho = 0.75$ it already misses the SLO at 240 ms, and by $\rho = 0.90$ a request waits more than half a second. The values match the module's quoted rules of thumb: at $\rho = 0.75$ the wait is 3× service; at $\rho = 0.90$, 9×.

**Mini-example — $\rho = 0.75$, the operation in isolation**

$\mathrm{E}[W] = 60 \times \dfrac{0.75}{0.25} = 60 \times 3 = 180$ ms. Add the 60 ms of service and TTFT = 240 ms — 40 ms over the 200 ms SLO. The cluster is only three-quarters loaded and already failing the interactive contract.

## 4 Step 3 — Where exactly does FIFO break?

Rather than read the wall off a table, solve for it. TTFT under FIFO is $t_{\mathrm{svc}}/(1-\rho)$; set it equal to the SLO and invert.

**The SLO-breaking utilization $\rho$**

$$\frac{t_{\mathrm{svc}}}{1-\rho} = \mathrm{SLO} \implies 1-\rho = \frac{t_{\mathrm{svc}}}{\mathrm{SLO}} \implies \rho = 1 - \frac{t_{\mathrm{svc}}}{\mathrm{SLO}}.$$

$$\rho = 1 - \frac{60}{200} = 1 - 0.30 = \mathbf{0.70}. \quad ✓$$

So FIFO holds the 200 ms SLO only up to $\rho = 0.70$. Above that, interactive TTFT exceeds budget — precisely the **utilization wall near** $\rho \sim 0.7$–$0.8$ the module warns about. The fix is not more compute; it is *priority scheduling*, which lets interactive requests jump the queue so their wait $\approx t_{\mathrm{svc}}$ independent of $\rho$, while batch requests absorb the full delay.

**Plan for $\rho \le 0.7$, or schedule by priority**

Under FIFO, target steady-state utilization at or below $\rho = 1 - t_{\mathrm{svc}}/\mathrm{SLO}$. For Nexus that is 0.70 — you must leave 30% of capacity idle as queueing headroom. The alternative to buying idle capacity is SLO-aware (priority / EDF) scheduling, which decouples interactive latency from $\rho$. That is why every production serving stack ships a priority scheduler, not FIFO.

**Watch out**

$\rho$ depends only on the *ratio* $t_{\mathrm{svc}}/\mathrm{SLO}$. A tighter SLO or a slower service time pushes the wall *down*: halve the SLO to 100 ms and $\rho = 1 - 60/100 = 0.40$ — you would need to run the cluster 60% idle under FIFO. This is why interactive and batch traffic cannot share a FIFO queue: their SLO ratios demand incompatible utilization ceilings.

## 5 Step 4 — The preemption threshold

The second cliff is memory, not time. Under continuous batching the scheduler keeps up to $S_{\max}$ sequences in flight, each holding a KV cache. If their combined KV approaches the pool, the scheduler must *preempt* — evict a running sequence, save or recompute its state later. Preemption is wasted work, so we want to know how close to the edge we sit.

**Preemption threshold**

$$\text{expected\_kv\_demand} = S_{\max} \cdot \mathrm{E}[T_{\text{ctx}}] \cdot \text{kv}_{\text{tok}}, \qquad \text{threshold} = \frac{\text{expected\_kv\_demand}}{M_{\text{kv}}}.$$

**Plug in the Nexus numbers**

$$\begin{aligned}\text{expected\_kv\_demand} &= S_{\max} \cdot \mathrm{E}[T_{\text{ctx}}] \cdot \text{kv}_{\text{tok}} = 64 \times 2000 \times 327680\,\text{B} \\ &= 4.194 \times 10^{10}\,\text{B} = 41.9\,\text{GB},\end{aligned}$$

$$\text{threshold} = \frac{41.9\,\text{GB}}{100\,\text{GB}} = \mathbf{0.419}. \quad \checkmark$$

A threshold of 0.42 is comfortably **healthy** — the system uses under half its KV pool at full concurrency, leaving 2.4× headroom for context-length variance.

| Threshold | Band | Behavior |
|---|---|---|
| $< 0.85$ | **Healthy** | room for variance; preemption rare |
| 0.85–0.92 | **Warning** | preemption rate 1–5% |
| $> 0.92$ | **Broken** | preemption storm; TPOT unstable |

## 6 Step 5 — Context length is the real variable

The demand grows *linearly* with mean context length. Holding $S_{\max} = 64$ and $M_{\text{kv}} = 100\,\text{GB}$, sweep $\mathrm{E}[T_{\text{ctx}}]$:

| $\mathrm{E}[T_{\text{ctx}}]$ | demand (GB) | threshold | verdict |
|---|---|---|---|
| 2000 | 41.9 | 0.419 | Healthy |
| 4000 | 83.9 | 0.839 | Healthy (edge) |
| 8000 | 167.8 | 1.678 | Broken — exceeds pool |
| 12000 | 251.7 | 2.517 | Broken |
| 16000 | 335.5 | 3.355 | Broken |

At 4 K mean context the pool is 84% full — right at the warning edge. At 8 K the demand (168 GB) *exceeds the entire pool*: the system cannot hold 64 sequences at all and preempts constantly. The lever is not the scheduler; it is $S_{\max}$ or the hardware.

**Preemption is a capacity problem, not a policy problem**

The preemption threshold rises linearly with context length and with $S_{\max}$. When it crosses 0.85, no scheduling policy can avoid evictions — the requests simply do not fit. The fixes are structural: lower $S_{\max}$, add KV pool (more or bigger GPUs), or shrink $\text{kv}_{\text{tok}}$ (FP8 KV, more aggressive GQA). Tuning the queue order cannot create memory that is not there.

**Watch out**

The two cliffs interact. Lowering $S_{\max}$ to tame the preemption threshold (Step 4) reduces the batch size, which lowers service throughput $\mu$, which raises $\rho$ for the same $\lambda$ — pushing you toward the *utilization* wall of Step 3. Capacity planning is the joint problem of staying left of both walls at once.

## 7 The argument, at a glance

| # | Step | Result |
|---|---|---|
| 1 | M/M/1 wait | $\mathrm{E}[W] = t_{\mathrm{svc}}\,\rho/(1-\rho)$; hyperbolic in $\rho$ |
| 2 | Wall sweep | $\rho$=0.75 ⇒ TTFT 240 ms, misses 200 ms SLO |
| 3 | Break point | $\rho = 1 - t_{\mathrm{svc}}/\mathrm{SLO} = 0.70$ ✓ |
| 4 | Preemption | threshold $= S_{\max}\mathrm{E}[T_{\mathrm{ctx}}]\mathrm{kv}/M_{\mathrm{kv}} = 0.42$ ✓ |
| 5 | Stress | 8 K context ⇒ threshold 1.68, preemption storm |

### 7.1 Equation cheat-sheet

| | |
|---|---|
| Utilization | $\rho = \lambda/\mu$ |
| M/M/1 wait | $\mathrm{E}[W] = t_{\mathrm{svc}} \cdot \rho/(1-\rho)$ |
| TTFT | $\mathrm{E}[W] + t_{\mathrm{svc}} = t_{\mathrm{svc}}/(1-\rho)$ |
| SLO break point | $\rho = 1 - t_{\mathrm{svc}}/\mathrm{SLO}$ |
| Expected KV demand | $S_{\max} \cdot \mathrm{E}[T_{\mathrm{ctx}}] \cdot \mathrm{kv}_{\mathrm{tok}}$ |
| Preemption threshold | $\mathrm{demand}/M_{\mathrm{kv}}$; healthy $< 0.85$ |
| KV per token | $2\,H_{\mathrm{kv}}\,d_{\mathrm{head}}\,N_{\mathrm{layers}}\,D_{\mathrm{bytes}}$ |

## Appendix A — Reference implementation

Every number above is produced by the engine script `m05_t04_scheduling_queue_engine.py`. It sweeps the M/M/1 wait against the SLO, solves for $\rho$, and computes the preemption threshold across context lengths.

```
"""
Reference implementation for: Module 05 - Topic 4
"SLO-Aware Scheduling - the utilization wall and the preemption threshold."

Two pieces of operational math:
  (1) M/M/1 expected wait  E[W] = service_time * rho/(1-rho),  rho = lambda/mu.
      As rho -> 1 the wait blows up: this is the utilization wall.
  (2) Preemption threshold = expected_kv_demand / kv_pool_bytes, with
      expected_kv_demand = max_num_seqs * E[context_tokens] * kv_bytes_per_token.

Every number printed here is transcribed into the worked-by-hand PDF.
"""

# ---------------------------------------------------------------------
# 0. The workload: a mixed-SLO Nexus cluster.
# ---------------------------------------------------------------------
mu          = 1000.0   # service capacity, requests/sec
service_ms  = 60.0     # mean service time for an interactive request (ms)
slo_ms      = 200.0    # interactive TTFT SLO (ms)

```

```
# ------------------------------------------------------------------------
# 1. M/M/1 expected wait as utilization rho rises.
#    E[W] = service_time * rho/(1-rho).
# ------------------------------------------------------------------------
def E_wait_ms(rho):     return service_ms * rho / (1 - rho)
def ttft_ms(rho):       return E_wait_ms(rho) + service_ms   # wait + service

print("== 1. The utilization wall (M/M/1, FIFO) ==")
print(f"service_time = {service_ms:.0f} ms, SLO = {slo_ms:.0f} ms")
print(f"{'rho':>6} {'lambda':>9} {'rho/(1-rho)':>12} {'E[wait]ms':>10} "
      f"{'TTFT ms':>9} {'SLO?':>5}")
for rho in [0.50, 0.70, 0.75, 0.80, 0.90, 0.95, 0.99]:
    lam = rho * mu
    w   = E_wait_ms(rho)
    t   = ttft_ms(rho)
    ok  = "OK" if t <= slo_ms else "MISS"
    print(f"{rho:>6.2f} {lam:>9.0f} {rho/(1-rho):>12.2f} {w:>10.1f} "
          f"{t:>9.1f} {ok:>5}")

# ------------------------------------------------------------------------
# 2. The wall near rho ~ 0.7-0.8: where does FIFO break the 200 ms SLO?
#    TTFT(rho) = service*(1 + rho/(1-rho)) = service/(1-rho).
#    Solve service/(1-rho) = SLO  ->  rho* = 1 - service/SLO.
# ------------------------------------------------------------------------
rho_break = 1 - service_ms / slo_ms
print("\n== 2. Where FIFO breaks the SLO ==")
print(f"TTFT(rho) = service/(1-rho); set = SLO -> rho* = 1 - service/SLO")
print(f"rho* = 1 - {service_ms:.0f}/{slo_ms:.0f} = {rho_break:.3f}")
print(f"Above rho = {rho_break:.2f}, FIFO interactive TTFT exceeds the {slo_ms:.0f} ms SLO.")
print("This is the 'utilization wall' near rho ~ 0.7-0.8 the module warns about.")

# ------------------------------------------------------------------------
# 3. Preemption threshold (KV-pool pressure).
#    expected_kv_demand = max_num_seqs * E[context] * kv_bytes_per_token
#    threshold = expected_kv_demand / kv_pool_bytes
# ------------------------------------------------------------------------
n_kv_heads = 8; head_dim = 128; n_layers = 80; bytes_fp16 = 2
kv_bytes_per_token = 2 * n_kv_heads * head_dim * n_layers * bytes_fp16  # 320 KB

max_num_seqs   = 64
E_context      = 2000          # mean context tokens per request
kv_pool_bytes  = 100e9         # KV budget pool-wide (bytes), illustrative

expected_kv_demand = max_num_seqs * E_context * kv_bytes_per_token
threshold = expected_kv_demand / kv_pool_bytes

def health(t):
    if t < 0.85:  return "HEALTHY (room for variance)"
    if t < 0.92:  return "WARNING (preemption 1-5%)"
    return "BROKEN (preemption storm)"

print("\n== 3. Preemption threshold ==")
print(f"kv_bytes_per_token = {kv_bytes_per_token} B = {kv_bytes_per_token/1024:.0f} KB")
print(f"expected_kv_demand = {max_num_seqs} * {E_context} * {kv_bytes_per_token} B")
print(f"                   = {expected_kv_demand:.3e} B = {expected_kv_demand/1e9:.1f} GB")
print(f"kv_pool            = {kv_pool_bytes/1e9:.0f} GB")
print(f"threshold = demand / pool = {threshold:.3f}  -> {health(threshold)}")

# ------------------------------------------------------------------------
# 4. Stress: longer contexts push the threshold toward the cliff.
# ------------------------------------------------------------------------
print("\n== 4. Threshold vs mean context length ==")
print(f"{'E[ctx]':>8} {'demand GB':>10} {'threshold':>10} {'verdict':>30}")
```

```
for ctx in [2000, 4000, 8000, 12000, 16000]:
    dem = max_num_seqs * ctx * kv_bytes_per_token
    th  = dem / kv_pool_bytes
    print(f"{ctx:>8} {dem/1e9:>10.1f} {th:>10.3f}   {health(th)}")
```

— end of document —